



UNIVALI

Universidade do Vale do Itajaí
Escola do Mar, Ciência e Tecnologia - EMCT
Ciência da Computação

SWAGGER

Gustavo Henrique Stahl Müller, Paulo Henrique Rohling
e Vinícius Gabriel Martins.



Sumário

- Definição da Ferramenta
- Tipos de Testes Suportados
- Situações de Uso
- Exemplos
- Vantagens e Desvantagens

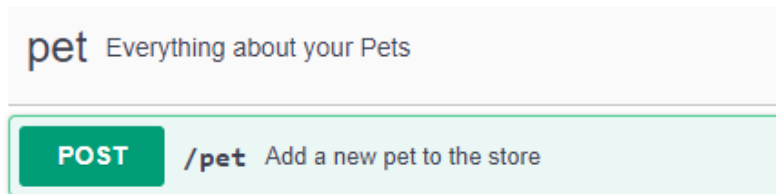
Definição da Ferramenta

- O Swagger é um framework para testes/documentação de APIs REST.
- A documentação gerada por ele a partir de um arquivo YAML/JSON é uma página web onde pode-se facilmente descobrir e entender as capacidades de uma API sem precisar acessar código fonte e demais documentações, possibilitando até testar requisições na página.
- Nessa apresentação falaremos sobre o Swagger Editor, Swagger Inspector e Swagger UI.

Funcionamento



- A API REST é descrita em YAML, mas pode ser convertida para *.json*.
- O Swagger Editor automaticamente gera a documentação da API (arquivos HTML) a partir do arquivo YAML ou JSON.
- Por último, o Swagger UI (pacote do NPM), permite encapsular toda a documentação em um só elemento HTML.

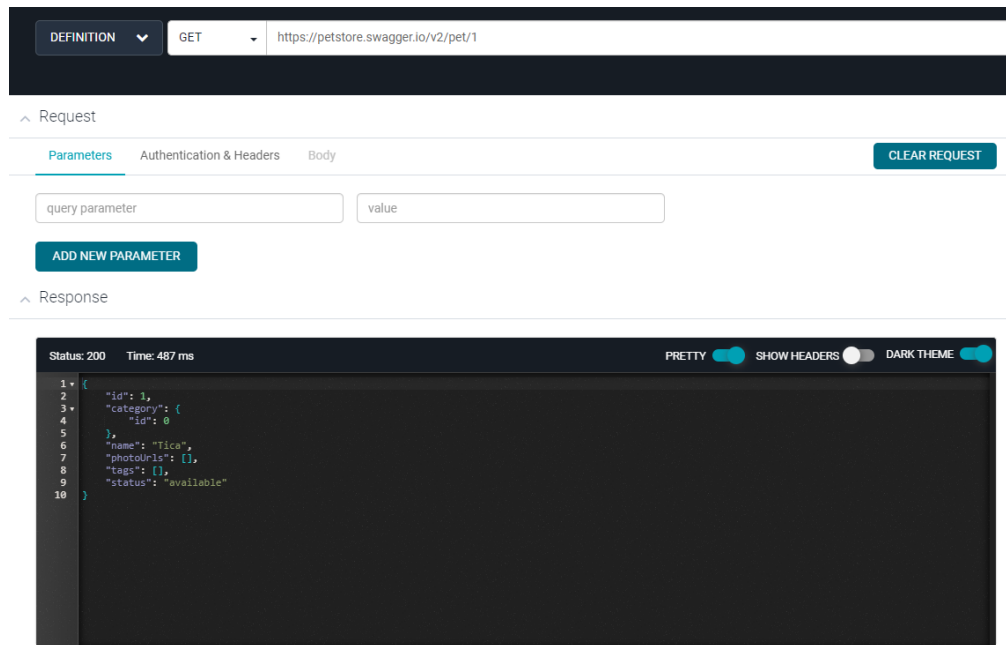
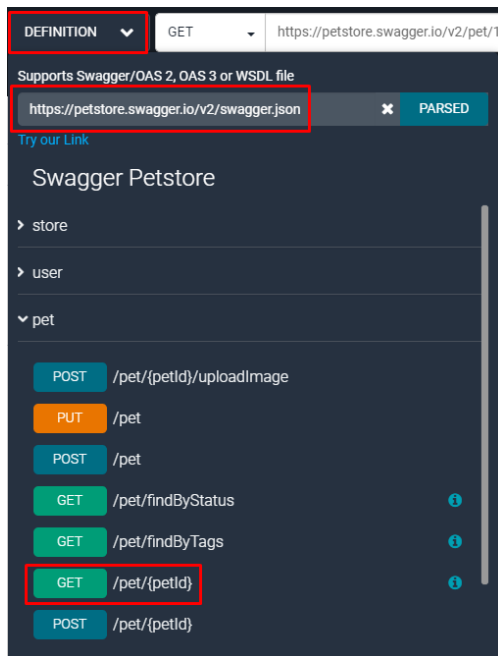


```
tags:
- name: pet
  description: Everything about your Pets
  externalDocs:
    description: Find out more
    url: http://swagger.io
- name: store
  description: Access to Petstore orders
- name: user
  description: Operations about user
```

```
paths:
  /pet:
    post:
      tags:
      - pet
      summary: Add a new pet to the store
      operationId: addPet
```

Testando a API

- É possível testar a API com o Swagger Inspector.



Testando a API

- Como deixar os testes automáticos?
- O Swagger não suporta testes automáticos por padrão; Apenas testes manuais, como vimos no slide passado.
- Porém, existem outras ferramentas que integram com o Swagger e permitem a criação automática de testes. Uma delas é o Assertible, que será usada para fins de exemplos.

- É possível importar um arquivo YAML ou JSON do Swagger para o Assertible.
- Testes automáticos são criados para todos os endpoints e métodos fornecidos pela API descrita no arquivo.
- Assertible permite a criação de *setups* e *teardowns* para cada teste, e também a criação de *assertions*.
- Por último, permite a execução de testes em conjunto.

- É possível criar *assertions* sobre: Códigos de resposta, valores do JSON de resposta, tempo de execução do teste, entre outros.

Variables ⓘ

{{username}} → Gustavo

Configure

Request Setups Teardowns **Assertions** Settings

☒ ● Status code ⓘ

Assert status code equals 200

ⓘ

☒ Automatically run test when saving assertions

Created 8 minutes ago · Updated a minute ago

Get user by user name GET /v... 404 status code · 401 ms

1 failed assertions

FAIL Status code equals 200
Reason: Status code 404 (Not Found) does not equal 200

CORE Valid SSL - TLS/SSL validated

CORE HTTP request success - Connected to host

CORE Valid environment - Loaded environment 'production'

Response **Request** Setups Teardowns

GET https://petstore.swagger.io/v2/user/Gusta... run_jankvha · 5 minutes ago

Body Headers

application/json

52 B · 📄 🔄

▼ object
code: 1
type: "error"
message: "User not found"

- Por último, é possível integrar os testes do Assertible com o Github, para que os testes sejam automaticamente executados após o *deployment* ocorrer.
- Uma solução mais fácil é simplesmente executar um comando CURL (gerado pelo Assertible) quando o *deployment* ocorrer:

```
curl -u $ASSERTIBLE_API_TOKEN: -XPOST https://assertible.com/deployments -d'{  
  "service" : "'${ASSERTIBLE_SERVICE_ID}'",  
  "environment" : "production",  
  "version" : "v1"  
}'
```

Desenvolvimento da documentação

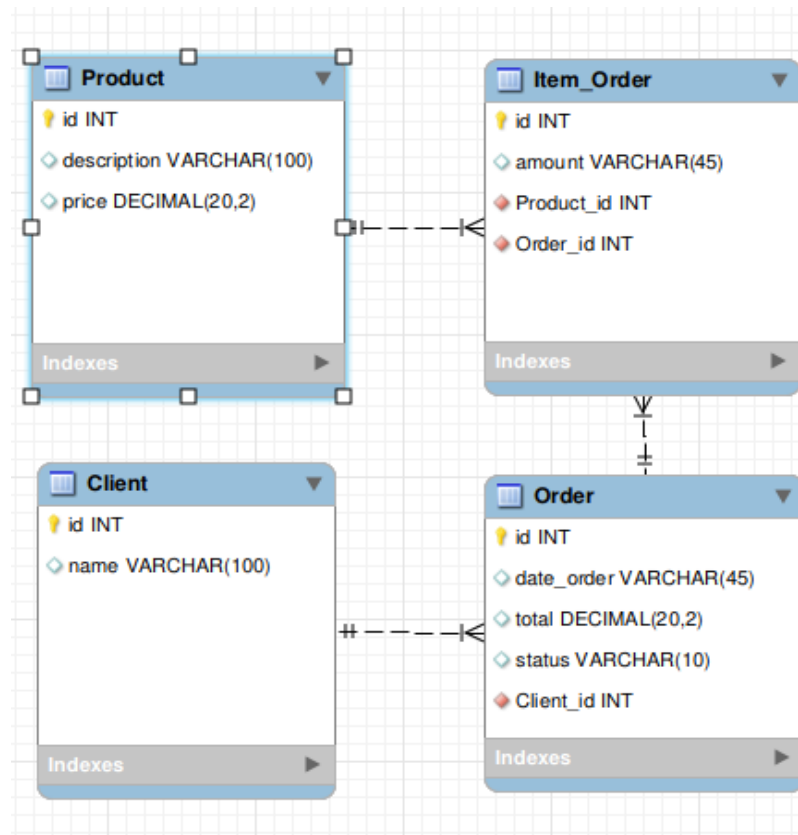


O Swagger permite criar a documentação da API de 3 formas:

- Automaticamente:
Simultaneamente ao desenvolvimento da API é gerada a documentação.
- Manualmente:
Escrevendo manualmente e publicando posteriormente.
- Codegen
Convertendo as anotações no código da API REST em documentação.

Exemplos

- Sistema de Vendas
- Aplicação REST
- Spring Boot
- MySQL
- ~~JWT~~



Exemplos

```
package org.rohling.rest.controller;

import ...

@RestController
@RequestMapping("/api/clients")
public class ClientController {

    @Autowired
    private ClientRepository clientRepository;

    @GetMapping("/{id}")
    public Client getClientById(@PathVariable("id") Integer id) {...}

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Client save(@RequestBody @Valid Client client) { return clientRepository.save(client); }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Integer id) {...}

    @PutMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void update(@PathVariable Integer id, @RequestBody @Valid Client client) {...}

    @GetMapping
    public List<Client> find(Client filter) {...}
}
```

```
package org.rohling.rest.controller;

import ...

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @Autowired
    private OrderService orderService;

    @PostMapping
    @ResponseStatus(CREATED)
    public Integer save(@RequestBody @Valid OrderDTO dto) {...}

    @GetMapping("/{id}")
    public OrderInformationsDTO getById(@PathVariable Integer id) {...}

    @GetMapping
    public List<Order> getAllOrders() { return orderService.getAllOrders(); }

    @PatchMapping("/{id}")
    @ResponseStatus(NO_CONTENT)
    public void updateStatus(@PathVariable Integer id, @RequestBody OrderStatusUpdateDTO dto) {...}

    private OrderInformationsDTO convert(Order order) {...}

    private List<ItemOrderInformationsDTO> convert(List<ItemOrder> itens) {...}
}
```

Exemplos

```
package org.rohling.rest.controller;

import ...

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping
    public List<Product> getAllProducts(Product filter) {...}

    @GetMapping("/{id}")
    public Product getProductById(@PathVariable("id") Integer id) {...}

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public void save(@RequestBody @Valid Product product) { productRepository.save(product); }

    @PutMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void update(@PathVariable Integer id, @RequestBody @Valid Product product) {...}

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Integer id) {...}

}
```

```
{
  "id": 1,
  "name": "PAULO"
},
{
  "id": 2,
  "name": "GUSTAVO"
},
{
  "id": 3,
  "name": "VINICIUS"
}
```

Exemplos

- Swagger Dependency
- Swagger UI

```
<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger2</artifactId>
  <version>2.9.2</version>
</dependency>

<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger-ui</artifactId>
  <version>2.9.2</version>
</dependency>
```

Exemplos

```
@Configuration
@EnableSwagger2
public class SwaggerConfig {

    @Bean
    public Docket docket() {
        return new Docket(DocumentationType.SWAGGER_2)
            .useDefaultResponseMessages(false) Docket
            .select().apis(RequestHandlerSelectors.basePackage("org.rohling.rest.controller"))
            .paths(PathSelectors.any()).build() Docket
            .securityContexts(Arrays.asList(securityContext()))
            .securitySchemes(Arrays.asList(apiKey()))
            .apiInfo(apiInfo());
    }

    private ApiInfo apiInfo() {
        return new ApiInfoBuilder()
            .title("Vendas API")
            .description("Api do projeto de Vendas")
            .version("1.0")
            .contact(contact())
            .build();
    }
}
```

Vendas API ^{1.0}

[Base URL: localhost:8080/]

<http://localhost:8080/v2/api-docs>

Api do projeto de Vendas

[Paulo H. Rohling - Website](#)

[Send email to Paulo H. Rohling](#)

<http://localhost:8080/swagger-ui.html#/>

Authorize



client-controller Client Controller



order-controller Order Controller



product-controller Product Controller



user-controller User Controller



Models



client-controller Client Controller



GET /api/clients find



POST /api/clients save



GET /api/clients/{id} getClientById



PUT /api/clients/{id} update



DELETE /api/clients/{id} delete



order-controller Order Controller



GET /api/orders getAllOrders



POST /api/orders save



GET /api/orders/{id} getById



PATCH /api/orders/{id} updateStatus



product-controller Product Controller



GET /api/products getAllProducts



POST /api/products save



GET /api/products/{id} getProductById



PUT /api/products/{id} update



DELETE /api/products/{id} delete



client-controller Client Controller



GET /api/clients find



Parameters

Cancel

Name

Description

id
integer(\$int32)
(query)

id

name
string
(query)

name

Execute

Clear

Responses

Response content type

/



Curl

```
curl -X GET "http://localhost:8080/api/clients" -H "accept: */*"
```

Request URL

```
http://localhost:8080/api/clients
```

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "name": "PAULO"
  },
  {
    "id": 2,
    "name": "GUSTAVO"
  },
  {
    "id": 3,
    "name": "VINICIUS"
  }
]
```

Download

Models

```
Client ▾ {  
  id  
  name  
}  
integer($int32)  
string
```

```
CredentialDTO ▾ {  
  login  
  password  
}  
string  
string
```

```
ItemOrder ▾ {  
  amount  
  id  
  order  
  product  
}  
integer($int32)  
integer($int32)  
Order > {...}  
Product > {...}
```

```
ItemOrderDTO ▾ {  
  amount  
  product  
}  
integer($int32)  
integer($int32)
```

```
ItemOrderInformationsDTO ▾ {  
  amount  
  description  
  price  
}  
integer($int32)  
string  
number
```

```
Order ▾ {  
  client  
  id  
  itens  
  orderDate  
  price  
  status  
}  
Client > {...}  
integer($int32)  
> [...]  
string($date)  
number  
string  
Enum:
```

Latest Supported Version

users



GET

/learn/api/public/v1/users

Get Users



POST

/learn/api/public/v1/users

Create User



GET

/learn/api/public/v1/users/{userId}

Get User



Loads a user.

Properties returned will depend on the caller's entitlements.

Callers with the entitlement 'user.VIEW' will always get a minimal set of core properties:

- id
- name.given
- name.family
- name.middle
- systemRoleIds

Depending on the Learn administrator customizable user field display settings, as well as the target user's personal visibility settings, the following fields might also be included:

- userName
- educationLevel
- gender
- birthDate
- name.other
- name.suffix
- name.title
- job.*
- contact.*
- address.*

Callers asking for their own user record get all fields listed above, plus:

- locale.*

All user properties are provided to callers with one of the entitlements 'system.user.properties.MODIFY', 'system.useradmin.generic.VIEW', or 'system.user.VIEW'.

Vantagens e Desvantagens



- ⊕ Comunidade grande gerando diversas ferramentas para o Swagger.
- ⊕ Framework multi-linguagem, implementado nas mais diversas tecnologias.
- ⊕ Interface gráfica auxilia no entendimento das APIs REST.
- ⊖ A configuração pode ser trabalhosa.
- ⊖ Não faz testes automáticos nativamente.

Referências



- **Blackboard REST API.** Disponível em: <<https://developer.blackboard.com/portal/displayApi>>
- **What is Swagger?.** Disponível em: <<https://swagger.io/docs/specification/2-0/what-is-swagger/>>
- **Documentando sua API Rest com Swagger.** Disponível em:
<<http://www2.decom.ufop.br/terralab/documentando-sua-api-rest-com-swagger/>>
- **Swagger (software).** Disponível em: <[https://en.wikipedia.org/wiki/Swagger_\(software\)](https://en.wikipedia.org/wiki/Swagger_(software))>
- **Swagger - Documente seu ASP.NET Web API Rest.** Disponível em:
<<https://rafaelcruz.azurewebsites.net/2016/08/23/swagger-documente-sua-api-rest/>>
- **Swagger - Spring REST Example.** Disponível em: <<https://howtodoinjava.com/swagger2/swagger-spring-mvc-rest-example/>>